Snakes & Ladders - How it works

Home notes + DESIGN.md + source code | compact print

In one line - Players take turns rolling dice, move on a serpentine board, snakes send you down, ladders up, exact land to win.

Story - how this LLD works

Game builds an N x N board (size should be a perfect square). Snakes and ladders are placed randomly via a factory. Players sit in a queue. Roll, move, apply obstacle, if you go past the end you stay put. Land exactly on the last cell to win and leave the queue.

Who does what

Game: The host - setup, turn loop, winner print.
Board: Cells + serpentine numbering + obstacles.
Cell: A position; maybe has a snake or ladder.
Player: Name + current position.
Dice: One or more dice summed.
Obstacle / Snake / Ladder: Jump from src to dest.
ObstacleFactory: Create snake or ladder from type.

Main pieces

Game loop

- deque of players
- roll -> move -> win?

Board

- serpentine cells
- obstacles

Snake

- head -> tail down

Ladder

- bottom -> top up

Step-by-step at runtime

1. Input size, snake count, ladder count, players, dice count.
2. Board created; obstacles placed without overlapping cells.
3. Turn: pop player from deque, roll dice, compute new position.
4. If overshoot -> stay, append back to queue.
5. Else move; if cell has obstacle, jump to dest.
6. If position == size -> win (do not re-queue). Else re-queue.

Why this design (plain words)

Why Factory? - Board placement code shouldn't care Snake vs Ladder construction details.

Why polymorphism? - Cell just asks obstacle for final position - same for snake or ladder.

Remember

- Must land exactly on last cell.
- Ladder constructor args are easy to mix up (top/bottom) - read the class once.
- Perfect square board size is assumed.

OOP in this project

Encapsulation - Board hides how row/col mapping works.

Abstraction - Obstacle means 'jump somewhere'.

Inheritance - Snake and Ladder are Obstacles.

Polymorphism - Same move path applies any obstacle on the cell.

DESIGN.md - full file

Complete markdown from lld/snakes_and_ladder/DESIGN.md (Q&A for learning).

Full DESIGN.md (same file as in this folder) - DESIGN.md

Designing Snakes and Ladders - LLD Q&A

Run: python -m lld.snakes_and_ladder.main
Patterns: Factory · Inheritance/Polymorphism · Facade

Q1. What problem are we solving?

A. Design a Snakes & Ladders simulator with:
Configurable board size, snakes, ladders, dice count, players
Serpentine board numbering (like a real board)
Random obstacle placement
Turn-based play with dice rolls
Snake/ladder effects and exact-land-to-win rule

Q2. What are the core entities?

- Entity - Responsibility
- Board - NxN grid, serpentine numbering, obstacles, movement
- Cell - Position + optional obstacle; get_final_position()
- Player - Name + current position
- Dice - Roll 1-6 per die, sum for multi-dice
- Obstacle - ABC: Snake (head->tail), Ladder (bottom->top)
- Game - Facade - setup, turn loop, win detection

Q3. How is the package structured?

```
snakes_and_ladder/
|-- model/      # Board, Cell, Player, Dice, Obstacle, Snake, Ladder
|-- factory/    # ObstacleFactory
|-- service/    # Game
|-- enums/      # ObstacleType
```

Q4. Why Factory for obstacles?

A. Board placement code should not know concrete classes:

```
ObstacleFactory.create_obstacle(ObstacleType.SNAKE, up, down)  -> Snake
ObstacleFactory.create_obstacle(ObstacleType.LADDER, up, down) -> Ladder
```

Board treats both uniformly via polymorphism (move_player / final position).

Q5. How does the board layout work?

side_length = sqrt(size) - size should be a perfect square
Cells numbered serpentine: row 1 L->R, row 2 R->L, etc.
Linear position maps to (row, col) via _get_row / _get_col

Q6. What is the turn loop?

```
while more than one player in queue:
  player = players.popleft()
  roll = dice.roll()
  new_pos = board.get_new_position(player, roll)
  if overshoot (> size): stay, re-queue
  elif new_pos == size: WIN (do not re-queue)
  else: update position (apply snake/ladder), re-queue
```

Players are managed with a deque (round-robin).

Q7. How are obstacles placed?

```
random up ∈ [2, size], down ∈ [1, up-1]
-> ObstacleFactory.create -> board.add_obstacle
-> reject if src or dest cell already has an obstacle
```

Q8. How do you extend this design?

- Add... - How
- Custom layouts - Load snakes/ladders from config instead of random
- New obstacles - Teleporter/penalty extending Obstacle
- Player strategies - Human input vs AI vs weighted dice
- Exact-roll rule toggle - Config flag on overshoot behavior

Q9. Common interview follow-ups

Q. Overshoot?
Player stays put and is re-queued - must land exactly on last cell to win.
Q. Ladder constructor args?

Ladder(top, bottom) stores src=bottom, dest=top (args inverted vs snake).
Q. Infinite loops?
Placement rejects overlapping cells; discuss validating no cycles if loading custom boards.

Source code - lld/snakes_and_ladder/

16 Python files below (compact font to save pages). Read with the notes: find the class name from the cast, then open that file here.
- Tip: start at main.py, then service/, then model/ + strategy/.

FILE: main.py (23 lines)

```
 1| from .model.player import Player
 2| from .service.game import Game
 3|
 4|
 5| def main() -> None:
 6|     board_size = int(input("Enter the board size : "))
 7|     no_of_snakes = int(input("Enter the number of snakes : "))
 8|     no_of_ladders = int(input("Enter the number of ladders : "))
 9|     no_of_players = int(input("Enter the number of players : "))
10|     no_of_dice = int(input("Enter the number of Dice : "))
11|
12|     game = Game(board_size, no_of_ladders, no_of_snakes, no_of_dice)
13|
14|     for i in range(no_of_players):
15|         name = input(f"Enter the name of player {i + 1} : ")
16|         player = Player(name)
17|         game.add_player(player)
18|
19|     game.start_game()
20|
21|
22| if __name__ == "__main__":
23|     main()
```

FILE: service/__init__.py (3 lines)

```
 1| from .game import Game
 2|
 3| __all__ = ["Game"]
```

FILE: service/game.py (86 lines)

```
 1| import random
 2| from collections import deque
 3|
 4| from ..enums.obstacle_type import ObstacleType
 5| from ..factory.obstacle_factory import ObstacleFactory
 6| from ..model.board import Board
 7| from ..model.dice import Dice
 8| from ..model.obstacle import Obstacle
 9| from ..model.player import Player
10|
11|
12| class Game:
13|     def __init__(
14|         self, size: int, no_of_ladders: int, no_of_snakes: int, no_of_dice: int
15|     ) -> None:
16|         self._no_of_snakes = no_of_snakes
17|         self._no_of_ladders = no_of_ladders
18|         self._board = Board(size)
19|         self._players: deque[Player] = deque()
20|         self._dice = Dice(no_of_dice)
21|
22|         self._init_board_obstacles()
23|
24|     @property
25|     def no_of_snakes(self) -> int:
26|         return self._no_of_snakes
27|
28|     @property
29|     def no_of_ladders(self) -> int:
30|         return self._no_of_ladders
31|
32|     @property
33|     def board(self) -> Board:
34|         return self._board
35|
36|     @property
37|     def players(self) -> deque[Player]:
38|         return self._players
39|
40|     @property
41|     def dice(self) -> Dice:
42|         return self._dice
43|
44|     def _init_board_obstacles(self) -> None:
45|         self._generate_obstacles(self._no_of_snakes, ObstacleType.SNAKE)
46|         self._generate_obstacles(self._no_of_ladders, ObstacleType.LADDER)
47|
48|     def _generate_obstacles(self, count: int, type_: ObstacleType) -> None:
49|         rng = random.Random()
50|         size = self._board.size
51|
52|         while count > 0:
53|             up = rng.randint(2, size)
54|             down = rng.randint(1, up - 1)
55|
56|             obstacle: Obstacle = ObstacleFactory.create_obstacle(type_, up, down)
57|             if self._board.add_obstacle(obstacle):
58|                 count -= 1
59|
60|     def add_player(self, player: Player) -> None:
61|         self._players.append(player)
62|
```

```
63|     def start_game(self) -> None:
64|         self._board.print_board(self._players)
65|
66|         while len(self._players) > 1:
67|             curr_player = self._players.popleft()
68|             print("-----------------------------------")
69|
70|             dice_roll = self._dice.roll()
71|             print(f"{curr_player.name} rolled {dice_roll}")
72|
73|             new_position = self._board.get_new_position(curr_player, dice_roll)
74|
75|             if new_position == curr_player.position:
76|                 self._players.append(curr_player)
77|                 continue
78|
79|             curr_player.position = new_position
80|
81|             if new_position == self._board.size:
82|                 print(f"{curr_player.name} has won the game!")
83|             else:
84|                 self._players.append(curr_player)
85|
86|             self._board.print_board(self._players)
```

FILE: model/__init__.py (17 lines)

```
 1| from .board import Board
 2| from .cell import Cell
 3| from .dice import Dice
 4| from .ladder import Ladder
 5| from .obstacle import Obstacle
 6| from .player import Player
 7| from .snake import Snake
 8|
 9| __all__ = [
10|     "Board",
11|     "Cell",
12|     "Dice",
13|     "Ladder",
14|     "Obstacle",
15|     "Player",
16|     "Snake",
17| ]
```

FILE: model/board.py (99 lines)

```
 1| import math
 2| from collections import deque
 3| from typing import Deque
 4|
 5| from ..enums.obstacle_type import ObstacleType
 6| from .cell import Cell
 7| from .obstacle import Obstacle
 8| from .player import Player
 9|
10|
11| class Board:
12|     def __init__(self, size: int) -> None:
13|         self._size = size
14|         self._side_length = int(math.sqrt(size))
15|         self._grid: list[list[Cell]] = [
16|             [None] * self._side_length for _ in range(self._side_length)  # type: ignore[list-item]
17|         ]
18|
19|         position = 1
20|         left_to_right = True
21|
22|         for i in range(self._side_length - 1, -1, -1):
23|             if left_to_right:
24|                 for j in range(self._side_length):
25|                     self._grid[i][j] = Cell(position)
26|                     position += 1
27|             else:
28|                 for j in range(self._side_length - 1, -1, -1):
29|                     self._grid[i][j] = Cell(position)
30|                     position += 1
31|             left_to_right = not left_to_right
32|
33|     @property
34|     def size(self) -> int:
35|         return self._size
36|
37|     def _get_row(self, position: int) -> int:
38|         row = (position - 1) // self._side_length
39|         return self._side_length - 1 - row
40|
41|     def _get_col(self, position: int) -> int:
42|         row = self._get_row(position)
43|         col = (position - 1) % self._side_length
44|         return self._side_length - 1 - col if row % 2 == 0 else col
45|
```

(cont.) model/board.py

```
    def _get_cell(self, position: int) -> Cell:
        return self._grid[self._get_row(position)][self._get_col(position)]

    def add_obstacle(self, obstacle: Obstacle) -> bool:
        src_cell = self._get_cell(obstacle.src)
        dest_cell = self._get_cell(obstacle.dest)

        if src_cell.has_obstacle() or dest_cell.has_obstacle():
            return False

        src_cell.obstacle = obstacle
        return True

    def get_new_position(self, player: Player, offset: int) -> int:
        new_position = player.position + offset

        if new_position > self._size:
            print("You are going out of the board! Better luck next time!")
            return player.position

        cell = self._grid[self._get_row(new_position)][self._get_col(new_position)]
        final_position = cell.get_final_position()

        if final_position < new_position:
            print(f"Oops! Snake has bitten {player.name}")
        elif final_position > new_position:
            print(f"Congratulations! {player.name} moved up through a ladder")
        else:
            print(f"{player.name} moved from {player.position} to {new_position}")

        return final_position

    def print_board(self, players: Deque[Player]) -> None:
        print("\nCurrent Board State:")

        for i in range(self._side_length):
            for j in range(self._side_length):
                position = self._grid[i][j].position
                cell_content = str(position)

                if self._grid[i][j].has_obstacle():
                    obstacle = self._grid[i][j].obstacle
                    if obstacle.get_obstacle_type() == ObstacleType.SNAKE:
                        cell_content = f"?{obstacle.dest}"
                    else:
                        cell_content = f"?{obstacle.dest}"

                for player in players:
                    if player.position == position:
                        cell_content = player.name

                print(f"{cell_content:<8}", end="")
            print()
        print()
```

FILE: model/cell.py (16 lines)

```
from dataclasses import dataclass, field
from typing import Optional

from .obstacle import Obstacle


@dataclass
class Cell:
    position: int
    obstacle: Optional[Obstacle] = field(default=None)

    def has_obstacle(self) -> bool:
        return self.obstacle is not None

    def get_final_position(self) -> int:
        return self.obstacle.move_player() if self.has_obstacle() else self.position
```

FILE: model/dice.py (13 lines)

```
import random


class Dice:
    def __init__(self, no_of_dices: int) -> None:
        self._no_of_dices = no_of_dices
        self._random = random.Random()

    def roll(self) -> int:
        total = 0
        for _ in range(self._no_of_dices):
            total += self._random.randint(1, 6)
        return total
```

FILE: model/ladder.py (10 lines)

(cont.) model/ladder.py

```
from ..enums.obstacle_type import ObstacleType
from .obstacle import Obstacle


class Ladder(Obstacle):
    def __init__(self, top: int, bottom: int) -> None:
        super().__init__(bottom, top)

    def get_obstacle_type(self) -> ObstacleType:
        return ObstacleType.LADDER
```

FILE: model/obstacle.py (24 lines)

```
from abc import ABC, abstractmethod

from ..enums.obstacle_type import ObstacleType


class Obstacle(ABC):
    def __init__(self, src: int, dest: int) -> None:
        self._src = src
        self._dest = dest

    @property
    def src(self) -> int:
        return self._src

    @property
    def dest(self) -> int:
        return self._dest

    def move_player(self) -> int:
        return self._dest

    @abstractmethod
    def get_obstacle_type(self) -> ObstacleType:
        pass
```

FILE: model/player.py (7 lines)

```
from dataclasses import dataclass, field


@dataclass
class Player:
    name: str
    position: int = field(default=1)
```

FILE: model/snake.py (10 lines)

```
from ..enums.obstacle_type import ObstacleType
from .obstacle import Obstacle


class Snake(Obstacle):
    def __init__(self, head: int, tail: int) -> None:
        super().__init__(head, tail)

    def get_obstacle_type(self) -> ObstacleType:
        return ObstacleType.SNAKE
```

FILE: factory/__init__.py (3 lines)

```
from .obstacle_factory import ObstacleFactory

__all__ = ["ObstacleFactory"]
```

FILE: factory/obstacle_factory.py (14 lines)

```
from ..enums.obstacle_type import ObstacleType
from ..model.ladder import Ladder
from ..model.obstacle import Obstacle
from ..model.snake import Snake


class ObstacleFactory:
    @staticmethod
    def create_obstacle(type_: ObstacleType, up: int, down: int) -> Obstacle:
        if type_ == ObstacleType.SNAKE:
            return Snake(up, down)
        if type_ == ObstacleType.LADDER:
            return Ladder(up, down)
        raise ValueError("Invalid obstacle type")
```

FILE: enums/obstacle_type.py (6 lines)

```
from enum import Enum


class ObstacleType(Enum):
    SNAKE = "SNAKE"
    LADDER = "LADDER"
```

FILE: __init__.py (4 lines)

```
from .service.game import Game
from .model.player import Player

__all__ = ["Game", "Player"]
```

FILE: enums/__init__.py (3 lines)

```
1| from .obstacle_type import ObstacleType
2|
3| __all__ = ["ObstacleType"]
```